第 9 章：对话型 Agent 开发

版本: v2.5 (2026-03-23 全书完成) **作者**: 内容撰写专家（场景篇）
状态: review（待技术审核）
最后更新: 2026-03-23

本章涉及面试题

1. **如何保持多轮对话的上下文连贯性？**（涉及 9.2、9.4 节）
2. **如何设计 Agent 的人格设定？人格设定需要包含哪些要素？**（涉及 9.3 节）
3. **如何识别用户情绪异常？检测到不满时应该如何处理？**（涉及 9.6 节）
4. **对话型 Agent 的记忆如何分层管理？各层存储什么内容？**（涉及 9.2 节）
5. **如何处理对话中的指代消解（如「他」「这个」「刚才说的」）？**（涉及 9.4 节）

本章概述

学习目标: - 理解对话型 Agent 的核心挑战：上下文连贯性、个性化、情绪感知 - 掌握记忆管理策略和人格设定方法 - 能够设计多轮对话处理流程和情绪识别干预机制 - 理解对话型 Agent 的最佳实践与常见陷阱

核心知识点: - 对话型 Agent 场景分类与核心挑战 - 三层记忆管理策略（短期/长期/工作记忆） - 人格设定五要素与一致性维护 - 多轮对话处理（上下文窗口、指代消解、话题追踪） - 情绪识别方法与流程干预机制

9.1 需求分析

9.1.1 对话型 Agent 场景分类

对话型 Agent 根据应用场景可分为四类，每类有不同的设计重点：

场景类型	典型用例	设计重点	记忆策略
客服对话	解答问题、处理投诉、引导操作	准确性、规范性	短期记忆为主，记录用户问题历史
个人助理	日程管理、提醒、信息查询	效率、主动性	长期记忆存储用户习惯和偏好

场景类型	典型用例	设计重点	记忆策略
陪伴聊天	情感交流、闲聊、兴趣讨论	共情、个性化	长期记忆存储兴趣和关系历史
创意协作	头脑风暴、方案讨论、内容共创	开放性、引导性	工作记忆记录创意进展和设定

核心差异分析：- **客服场景**重准确性，回答必须与官方信息一致，不能自由发挥 - **助理场景**重效率，需要主动提醒和预测用户需求 - **陪伴场景**重共情，需要理解用户情绪并给予情感回应 - **创意场景**重开放性，需要引导用户发散思考而非给出标准答案

> **关键定义**：漫剧创意沟通 Agent 属于**创意协作场景**，需要开放性引导而非标准答案，记忆策略需要同时维护短期对话历史和长期用户偏好。

9.1.2 核心挑战：上下文连贯性

为什么上下文连贯性是对话型 Agent 的核心挑战?

LLM（大型语言模型）本身是**无状态**的，每次调用都是独立的计算过程，不记得之前的对话。这意味着：

```
问题根源：LLM 无状态特性
    │
    ├── 窗口限制：上下文窗口有限（8K/128K token），长对话需要截断
    │
    ├── 指代消解：用户说「他」「这个」「刚才说的」，需要解析指代对象
    │
    └── 话题追踪：对话可能多个话题交织，需要记录当前话题和切换历史
```

具体挑战分析：

1. **窗口限制**：主流模型上下文窗口从 8K 到 200K 不等，但成本随窗口增大而增加。实际应用中需要在成本和连贯性之间权衡。

2. **指代消解**：用户说「把刚才那个设定改一下」，Agent 需要知道「刚才那个」指哪个设定。这需要维护对话实体的提及历史。

3. **话题追踪**：对话可能在「角色设定」「剧情大纲」「世界观」之间切换，需要记录话题历史以支持「回到刚才的话题」这类请求。

> **注意**：认为「把全部历史塞进上下文就好」是常见误区。实际窗口有限且成本高，需要智能的记忆管理策略。

9.1.3 核心挑战：个性化

为什么需要个性化?

用户期望 Agent 记住自己的偏好、历史、习惯，而不是每次都重新介绍。

个性化维度：

维度	示例	实现方式
称呼偏好	「叫我老王」「别叫我先生」	长期记忆存储，对话时检索注入
回复风格	「喜欢简洁」「不要太啰嗦」	长期记忆存储，调整生成参数
兴趣标签	「玄幻迷」「不喜欢现代题材」	向量数据库存储，创意生成时检索
交互习惯	「早上活跃」「晚上不回复」	时间序列记录，调整推送策略

实现方式： 1. 长期记忆存储用户画像（**向量数据库**：支持语义检索的数据库） 2. 对话时检索 **Top-K**（最相关的 K 条）相关偏好注入上下文 3. 新信息写入、旧信息衰减（时间衰减权重） 4. 核心设定变更需用户确认

> **隐私边界**：哪些信息应该存储、哪些不应该、用户如何查看/删除自己的数据，这是个性化设计必须考虑的问题。建议提供用户设置界面，让用户明确知道存储了什么、可以删除什么。

9.1.4 本节小结

对话型 Agent 分客服/助理/陪伴/创意四类，核心挑战是**上下文连贯性**（窗口限制、指代消解、话题追踪）和**个性化**（用户画像、隐私边界），需要智能记忆管理策略。

9.2 方案设计：记忆管理策略

总：对话型 Agent 的记忆分为三层——短期记忆存对话历史、长期记忆存用户画像、工作记忆存任务状态，三层协同工作保证对话连贯性和个性化体验。

9.2.1 短期记忆（对话上下文）

定义：短期记忆存储最近 N 轮对话（用户输入 + Agent 回复），保证对话的即时连贯性。

实现方式对比：

方式	说明	适用场景	优缺点
ConversationBufferMemory	存储全部对话历史	短对话（<20 轮）	优点：完整；缺点：token 消耗大
ConversationBufferWindowMemory	只保留最近 N 轮	长对话场景	优点：可控；缺点：丢失早期信息
ConversationSummaryMemory	用 LLM 摘要早期对话	超长对话	优点：节省 token；缺点：摘要可能丢失细节

截断策略： 1. **固定窗口**：保留最近 10-20 轮对话 2. **Token 计数**：不超过 4K-8K token 3. **时间窗口**：保留最近 30 分钟的对话

优化技巧： - **摘要压缩**：当对话超过窗口时，用 LLM 摘要早期对话（「前 10 轮讨论了角色 A 和角色 B 的设定」） - **关键信息提取**：从早期对话提取事实性信息（设定、偏好）注入长期记忆，对话历史可丢弃

> **最佳实践**：漫剧创意沟通用**窗口记忆**保留最近 15 轮对话，超过后用 LLM 摘要早期内容保留关键设定，平衡连贯性和成本。

9.2.2 长期记忆（用户画像与历史）

定义：长期记忆存储用户偏好、历史交互、重要事实，支持语义检索和个性化注入。

实现架构：

```
┌──────────────────────────────────────────┐
│              长期记忆（向量数据库）              │
├──────────────────────────────────────────┤
│  用户偏好：「不要现代题材」「喜欢多主角」│
│  历史交互：第 1-5 章创意沟通记录              │
│  重要事实：作者是网络小说作家、玄幻迷      │
│  情绪历史：对「都市题材」有负面反馈        │
└──────────────────────────────────────────┘
```

```
                ↑                    ↑
    对话时检索 Top-K      新信息写入
    注入上下文              + 旧信息衰减
```

检索策略： - 对话时检索 Top-K 相关记忆（K=3-5）注入上下文 - 用**嵌入模型**（将文本转换为向量的模型）将用户表述向量化，检索语义相似的記憶 - 检索结果按相关性和时间衰减加权排序

更新机制： 1. **新信息写入**：用户明确表达的偏好直接写入 2. **旧信息衰减**：时间衰减权重（如 30 天前记忆权重降为 50%） 3. **用户确认**：核心设定变更需询问确认（「检测到您之前说喜欢玄幻，现在说不喜欢，是改变想法了吗？」）

> **常见误区**：长期记忆只存不更新。实际需要衰减和清理机制，避免过时信息干扰。例如用户 3 个月前说喜欢玄幻，但现在明确说不喜欢了，旧记忆应该衰减或被覆盖。

9.2.3 工作记忆（任务状态）

定义：工作记忆存储当前任务的中间状态、待办事项、进度追踪，支持任务中断恢复。

与短期记忆的区别：

维度	短期记忆	工作记忆
存储内容	对话历史（用户说了什么）	任务状态（做到哪一步了）
数据结构	文本列表（对话轮次）	结构化数据（JSON/Dict）
生命周期	对话结束可清除	任务完成才清除
用途	保证对话连贯性	支持任务中断恢复

实现方式： - 结构化存储（JSON/Dict），Agent 可读写更新 - 任务开始创建、任务完成清除、支持中断恢复

案例：漫剧生成任务工作记忆记录：

```
{
  "task": "漫剧大纲生成",
  "phase": "大纲生成",
  "progress": "3/5 章",
  "pending": ["角色 A 设定待确认"],
  "created_at": "2026-03-22T10:00:00Z"
}
```

> **关键区别**：短期记忆是「对话历史」，工作记忆是「任务状态」。例如短期记忆记录「用户说喜欢玄幻」，工作记忆记录「正在生成大纲，已完成 3/5 章」。

9.2.4 记忆冲突处理

问题：当新信息与旧记忆矛盾时，如何处理？

冲突类型： 1. **直接矛盾**：旧记忆「我喜欢玄幻」vs 新说「我不喜欢玄幻了」 2. **部分矛盾**：旧记忆「喜欢玄幻和仙侠」vs 新说「不喜欢仙侠」 3. **隐含矛盾**：旧记忆「不要现代题材」vs 新创意是现代题材

解决策略：

策略	说明	适用场景
时间优先	新信息覆盖旧信息	用户明确表达改变想法
来源优先	用户明确陈述优先于推断	明确表述 vs Agent 推断
置信度优先	计算综合置信度（来源/时间/一致性）	复杂冲突场景
用户确认	核心设定冲突时询问用户	重要设定变更

实现流程：

```
检测冲突 → 计算置信度 → 判断是否需确认 → 更新记忆 → 记录变更历史
    │              │                 │
    │              │                 └─ 核心设定 → 询问用户
    │              └─ 高置信度 → 直接更新
    └─ 相似度<0.5 → 判定为冲突
```

> **最佳实践**：漫剧系统检测到作者改变题材偏好时，询问确认「检测到您之前说喜欢玄幻，现在说不喜欢，是改变想法了吗？」，更新长期记忆并记录变更时间。

9.2.5 本节小结

记忆分三层：**短期记忆**存对话历史（窗口/摘要），**长期记忆**存用户画像（向量检索），**工作记忆**存任务状态（结构化）。冲突处理需要综合时间/来源/置信度判断，核心设定变更需用户确认。

9.3 方案设计：人格设定与一致性

总：人格设定包含角色定位、语言风格、知识边界、价值观、交互方式五要素，一致性需要持续注入和漂移检测，个性化适配在核心人格框架内调整。

9.3.1 人格设定五要素

完整的人格设定应包含以下五要素：

要素	说明	示例（漫剧创意 Agent）
角色定位	身份描述	「专业漫剧策划师」「资深编剧顾问」
语言风格	正式/casual、简洁/详细、幽默/严肃	「语言简洁直接，不用网络流行语」
知识边界	明确能做什么、不能做什么	「擅长创意策划，不擅长法律建议」
价值观	原则性声明	「尊重原创」「注重逻辑一致性」
交互方式	主动引导 vs 被动响应、提问频率	「主动提问澄清需求，每 3 轮确认一次进展」

> **常见误区**：人格设定过于笼统（如「友好助手」）。实际需要具体描述角色、风格、边界、价值观、交互方式五要素。

设计建议： 1. **角色定位**要具体，避免「助手」这类泛化描述 2. **语言风格**要有可操作的描述（如「回复不超过 200 字」） 3. **知识边界**要明确，避免用户产生错误预期 4. **价值观**要体现专业性（如「注重逻辑一致性」） 5. **交互方式**要量化（如「每 3 轮确认一次」）

9.3.2 人格一致性维护

为什么人格一致性重要?

人格跳跃会让用户困惑（如前一句专业后一句卖萌），降低信任感。

维护方法： 1. **系统提示词固定人格**：每次调用 LLM 都注入人格描述 2. **对话中持续注入**：长对话中定期重新注入人格描述（如每 10 轮） 3. **输出前检查一致性**：用嵌入模型计算当前回复与人格描述的余弦相似度

漂移检测机制：

```
生成回复 → 计算与人格描述的相似度 → 低于阈值告警 → 重新生成
                    |
                    └─ 阈值建议：0.75（**余弦相似度**：向量夹角余弦值，范围 -1 到 1）
```

修复机制： - 检测到人设漂移时重新注入系统提示词 - 或标记该回复，由人工审核 - 或自动重新生成直到符合人格

> **案例**：漫剧 Agent 检测到回复过于随意（相似度 0.6<阈值 0.75），重新生成保持专业风格。

9.3.3 个性化适配

问题：如何在保持人格一致性的同时适应用户偏好?

解决方案：在核心人格框架内调整非核心要素。

适配维度：

维度	可调整	不可调整
回复长度	✅ 可根据用户反馈调整	-
提问频率	✅ 可根据用户偏好调整	-
专业程度	⚠️ 可微调但不能突破边界	-
核心人格	-	❌ 专业 Agent 不能变成卖萌
价值观	-	❌ 不能违背原则

实现流程： 1. 记录用户对回复风格的反馈（如「太啰嗦」「不够详细」） 2. 根据反馈调整语言风格、回复长度、提问频率 3. 边界检查：确保调整不突破人格核心 4. 提供设置界面让用户明确选择偏好

> **最佳实践**：漫剧作者反馈「太啰嗦」，Agent 调整后续回复长度从 300 字降至 100 字，但保持专业风格（核心人格不变）。

9.3.4 本节小结

人格设定包含**角色/风格/边界/价值观/交互方式**五要素，一致性需要持续注入和漂移检测（相似度阈值 0.75），个性化适配在核心人格框架内调整（回复长度/提问频率可调，核心人格不可变）。

9.4 方案设计：多轮对话处理

总：多轮对话处理需要管理上下文窗口（截断 + 摘要 + 关键信息提取）、解析指代消解（规则/LLM/向量检索）、追踪话题切换（记录话题历史支持回溯）。

9.4.1 上下文窗口管理

窗口限制：主流模型上下文窗口从 8K 到 200K 不等，但成本随窗口增大而增加。

截断策略对比：

策略	说明	优点	缺点
固定轮数	保留最近 N 轮（N=10-20）	简单可控	可能丢失关键信息
Token 计数	不超过 X token（X=4K-8K）	精确控制成本	实现稍复杂
时间窗口	保留最近 N 分钟	符合时间直觉	对话密集时可能丢失

优化策略： 1. **摘要压缩**：用 LLM 摘要早期对话（「前 10 轮讨论了角色 A 和 B 的设定」） 2. **关键信息提取**：从早期对话提取事实性信息（设定、偏好）注入长期记忆，对话历史可丢弃

> **最佳实践**：漫剧长对话用 Token 计数限制 6K，超过后用 LLM 摘要早期内容保留设定信息，平衡连贯性和成本。

9.4.2 指代消解

问题：用户说「把他的能力改一下」，Agent 如何知道「他」指谁？

指代类型： 1. **人称代词**：他/她/它 2. **指示代词**：这个/那个/这些 3. **省略**：「生成大纲」「要哪种？」

消解方法对比：

方法	说明	准确率	成本
规则匹配	最近提及的实体优先	70-80%	低
LLM 推理	Prompt「这句话中的『他』指谁？」	85-90%	中
向量检索	将指代表述向量化，检索最相关实体	80-85%	中

实现技巧：- 在上下文中显式标注实体（「角色 A（张三）」），减少歧义 - 不确定时主动确认（「您说的是角色 A 吗？」）

> **常见误区**：依赖 LLM 自动消解。实际 LLM 可能猜错，最好显式确认。例如作者说「把他的能力改一下」，Agent 检索最近讨论的角色是「角色 A」，回复确认「您说的是角色 A 吗？」

9.4.3 话题追踪与切换

问题：对话可能在多个话题之间切换，如何追踪？

话题识别：- 用关键词或嵌入模型识别当前话题（如「角色设定」「剧情大纲」「世界观」） - 记录话题切换历史（「角色设定→剧情大纲→角色设定」）

话题内连贯：- 同一话题内保持术语一致、设定一致 - 话题内实体提及历史用于指代消解

话题切换处理：1. 检测切换时确认（「我们先放下角色设定，开始讨论剧情大纲对吗？」） 2. 记录切换点，支持「回到刚才的话题」请求 3. 切换时保存当前话题状态到工作记忆

> **最佳实践**：漫剧对话从「角色设定」切换到「剧情大纲」，Agent 确认切换并记录话题历史支持回溯。用户说「回到角色设定」时，Agent 能准确恢复到之前的话题状态。

9.4.4 本节小结

上下文窗口管理用**截断 + 摘要 + 关键信息提取**，指代消解用**规则/LLM/向量检索**（不确定时主动确认），话题追踪**记录切换历史**支持回溯。

9.5 最佳实践与陷阱

9.5.1 最佳实践

实践	说明	案例
记忆分层清晰	短期/长期/工作记忆职责明确，不混用	漫剧系统三层记忆独立存储
人格设定具体	角色/风格/边界/价值观/交互方式五要素完整	漫剧 Agent 人格文档 500 字
主动确认关键信息	指代消解、话题切换、设定变更时主动确认	「您说的是角色 A 吗？」

实践	说明	案例
情绪感知	连续 2 轮情绪分数<50 时，切换交互方式	检测不满时停止生成并询问
用户控制	提供设置界面让用户管理记忆、调整偏好	用户可查看/删除存储的偏好

9.5.2 常见陷阱

陷阱	后果	解决方案
记忆过载	存储太多细节导致检索噪声	只存关键信息，定期清理
人格漂移	长对话后人设不一致	持续注入人格描述，定期检测（阈值 0.75）
指代错误	猜错指代对象导致答非所问	主动确认而非猜测
情绪忽视	用户不满继续强行推进	情绪检测 + 流程干预（见 9.6 节）
隐私风险	存储敏感信息未加密	敏感信息加密存储，提供删除接口

> **案例**：漫剧 Agent 检测作者连续负面反馈后停止生成，询问「有什么需要改进的地方吗？」，避免加剧用户不满。

9.5.3 本节小结

最佳实践包括**记忆分层、人格具体、主动确认、情绪感知、用户控制**，常见陷阱是**记忆过载、人格漂移、指代错误、情绪忽视、隐私风险**。

9.6 情绪识别与流程干预

总：情绪识别需要综合文本分析、关键词和语气三种方法，阈值设定分级响应（正常>60/轻度负面 30-60/重度负面≤30），干预机制核心是 5 秒内停止生成并转向问题解决，情绪记忆管理 30 天内避免重复触发负面体验。

9.6.1 情绪识别方法

三种方法综合判断：

方法	说明	权重	示例
文本情绪分析	使用情绪分类模型（正面/负面/中性）	50%	调用百度 NLP API
关键词检测	建立负面情绪词库匹配	30%	「不满意」「太差了」「投诉」
语气分析	检测感叹号密度、反问句、重复表达	20%	连续 3 个感叹号、同一问题问 3 次

综合评分公式:

```
情绪分数 = 情绪分析 × 50% + 关键词 × 30% + 语气 × 20%
```

> **常见误区**：只依赖单一方法（如只用关键词）。实际需要综合判断，避免误报（如用户说「太好了」含「太」但不是负面）。

9.6.2 情绪阈值设定

分级响应机制:

级别	情绪分数	响应策略
正常	> 60	继续当前流程，无需特殊处理
轻度负面	30-60	主动询问「有什么需要改进的地方吗？」
重度负面	≤ 30	立即触发干预流程，停止当前任务

动态调整：根据用户历史情绪记录调整阈值（如某用户容易激动，阈值适当提高）。

> **案例**：漫剧生成系统设定轻度负面阈值为 50 分，检测到作者对大纲不满意时主动询问「您希望调整剧情节奏还是角色设定？」

9.6.3 流程干预机制

重度负面情绪的干预流程:

```
检测重度负面 → 停止生成 → 道歉与补救 → 转人工/降级
    │              │          │
    │              │          └─ 提供具体解决方案
```

```
    │                   └─ 保存进度，避免浪费 Token
    └─ 分数 ≤ 30 触发
```

干预策略：1. **中断当前任务**：立即停止生成，保存当前进度 2. **道歉与补救**：承认问题（「抱歉没有理解您的需求」），提供具体解决方案 3. **转人工客服**：严重负面情绪（分数 < 20）或连续 3 次轻度负面时，提供转接人工选项 4. **降级策略**：从自动生成切换到引导模式（让用户选择模板或提供示例）

> **最佳实践**：漫剧创意沟通检测到作者重度负面时，停止生成并回复「抱歉我的理解有偏差，您能否提供一个参考案例，或我为您转接人工策划师？」

9.6.4 情绪记忆管理

问题：如何避免重复触发用户负面情绪?

解决方案：记录用户情绪历史，生成内容前检索避开曾触发不满的方向。

实现方式：1. **记录情绪历史**：将每次交互的情绪分数存入长期记忆（向量数据库），标注触发负面情绪的主题 2. **避免重复触发**：生成内容前检索情绪历史，避开用户曾表达不满的方向 3. **个性化调整**：根据用户偏好调整交互方式 4. **情绪衰减**：旧的情绪记录随时间衰减权重（如 30 天前的负面记录权重降为 50%）

> **案例**：漫剧系统记录作者对「都市题材」的负面反馈，下次创作时优先询问「这次想尝试玄幻还是仙侠题材？」避开曾触发不满的方向。

9.6.5 本节小结

情绪识别需要**综合文本分析、关键词和语气**三种方法，阈值设定**分级响应**（正常>60/轻度负面30-60/重度负面≤30），干预机制核心是**5 秒内停止生成并转向问题解决**，情绪记忆管理**30 天内避免重复触发**负面体验。

9.7 简单举例

案例设计

- 案例名称：漫剧创意沟通 Agent
- 涉及知识点：对话型 Agent 的记忆管理、人格设定、多轮对话处理、情绪识别与流程干预
- 案例目标：帮助理解如何设计一个能够与作者流畅沟通漫剧创意的对话型 Agent
- 案例内容要点：
- 场景描述：作者与 Agent 沟通漫剧创意，多轮对话中讨论角色设定、剧情大纲、世界观

- 技术应用：短期记忆保留最近 15 轮对话并摘要早期内容，长期记忆存储作者偏好并检索注入，设定专业策划师人格，检测连续 3 次负面反馈时干预
- 效果说明：多轮对话连贯（指代消解准确率>90%）、人格一致（相似度>0.75）、情绪感知<10 秒响应，作者体验流畅
- 注意事项：不展开情绪识别的具体算法实现（见 9.6 节）

知识来源: - LangChain Memory 官方文档 - https://python.langchain.com/docs/modules/memory/ - AutoGen GroupChat 官方文档 - https://microsoft.github.io/autogen/docs/groupchat - ReAct 论文：Reasoning and Acting in Language Models (ICLR 2023) - https://arxiv.org/abs/2210.03629

修改记录: - v2.2 (2026-03-23): 量化指标 — 模糊表述改为具体数字（情绪阈值、响应时间、准确率） - v2.1 (2026-03-23): 术语定义 — 首次出现必定义（LLM、向量数据库、Top-K、嵌入模型、余弦相似度） - v2.0 (2026-03-23): 文字润色 — 句子简化、删除重复、优化结构 - v1.1 (2026-03-22): 根据编辑统筹意见修改 — 规范知识来源格式（2-3 个权威来源） - v1.0 (2026-03-22): 初稿完成